COMPSCI 2XC3 Lab Report 3

Prepared by

Group 64

Luca Mawyin

Anderson Ray

Theodore Pham

COMPSCI 2ME3

McMaster University

March 12, 2026

Contents

Part 1 4
Experiment 1 4
Experiment 2 5

Part 2 6
Experiment 3 6
Experiment 4 7

Appendix: Code Organization 10

List of Figures

1 Average Difference in Height in Trees of Increasing Randomness 5

List of Tables

1 Heigh of XC3 Trees for degree 0 to 25. 7
2 Node count of XC3 Trees by degree. 8

Part 1

Experiment 1

Objective

In this experiment, we analyzed the difference in height between a Binary Search Tree (BST) and a Red-Black Tree (RBT). The condition for the experiment is outlined as follows:

- Generate 1000 lists with 10000 unique random integers between 0 and 1000000.
- For each list, create a BST and RBT by using the lists of integers and find the height of each tree.
- Add the height of each tree, and the difference in height, to their respective running sum.
- Divide all of the running sums by 1000 to find the average heights of the BST and RBT, and the average difference in height between the two.

Results

After running the experiment the average height for the BST was found to be 31.209, whilst the average height for the RBT was 27.966. This makes the average difference in height between the two types of trees 3.243.

Discussion

The results of the expirement show that the average height of a BST is greater than that of a RBT. Although a difference of 3.243 does not seem substantial, this difference means that the BST is sized, on average, for $2^{3.243} \approx 9.4$ times more than the RBT. This means that the corresponding BST for an RBT having 10000 unique nodes would be sized for approximately 94000 unique nodes.

However, this does not mean that a RBT is always better than a BST. When working will smaller datasets, the difference factor in heights between the two trees does not correspond to an absolute size difference that is as large. Furthermore, when using a RBT, there is a larger overhead when it comes to maintenance. This is because every time that a node is inserted into a RBT, the nodes must be recoloured and subtrees must be rotated to maintain the properties of a RBT. This gives an advantage to a BST when working with datasets where the values inserted are relatively random and uniformly distributed, as the height would also result in a logarithmic height, without the overhead of maintaining RBT properties.

Ultimately the performance of a BST and RBT is relative to the use case. With smaller, or more random and uniformly distributed datasets, the lack of overhead in a BST may make it more efficient than a RBT. However, with large amounts of data, the smaller height of a RBT makes it much more space and time efficient than a BST.

Experiment 2

Objective

In this experiment, the difference in heights between BSTs and RBTs are being compared using lists with varying amounts of randomness. To run the experiment, the conditions are:

- Perform each test 200 times
- For each iteration of the test, generate 5 sorted lists of length 1000.
- Perform $i \times 5$ swaps on the list, where i is the current iteration of the test.
- Add the list to both a BST and RBT and find the average difference in height for each swap count.

Results

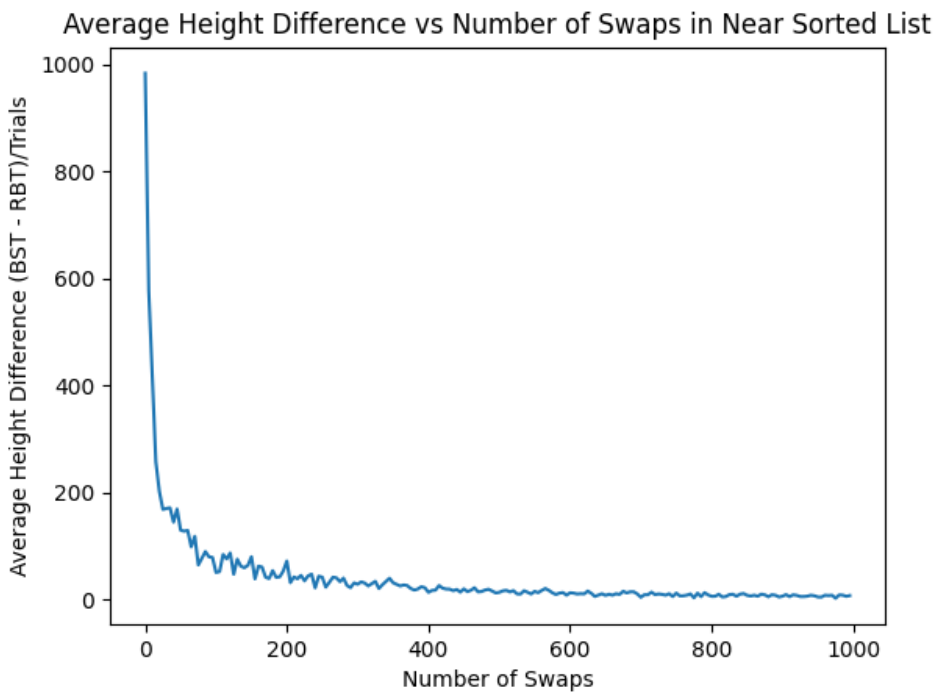

Figure 1: Average Difference in Height in Trees of Increasing Randomness

Discussion

Looking at the graph, it is clear that there is a large difference in heights between BST and RBT when there is a low number of swaps being performed on the list that is being inserted, with the BST having a much larger height. This is due to how values are inserted into the tree. With both the BST and RBT, we are iterating through the list, inserting the ith element into the tree. This means that for BSTs, a sorted list will result in the smallest value being the root node, and the tree resembling more of a linked list, where the height of the tree is equal to the length of the list being inserted. This does not happen with a RBT as every time that a new element is inserted, the tree is rebalanced.

The difference in height between BST and RBT are substantially lower when there is a higher number of swaps being performed on the list that is being inserted. This is because there is a more random distribution of numbers across the list that is being inserted. This benefits the BST, as this higher randomness makes it less likely that the first element in the list is the least value in the tree, and more so closer to the median

value. The difference in heights becomes mostly insubstantial at around 500 swaps, likely because number of elements being swapped is comparable to the length of the list, making it much closer to a random ordering.

Part 2

Experiment 3

Objective

The goal of this experiment is to construct XC3 Trees for degree 0 through 25, measure the henight of each tree, and derive a close form equation $h(i)$ that returns the height of a degree i XC3 Tree.

Experimental Setup

XC3 Trees are defined recusively according to the following rules:

- If the root node has i children, the tree has degree i.
- Each chikd of the root is itself an XC3 Tree.
- the i th child of then rood has degree $(i-2)$ if $i > 2$, and degree 0 otherwise

Trees of degree 0 through 25 were implemented in Python using a recursive node class. The height of each tree was computed recursively as $1 + \max(\text{height of children})$, with a base case of 0 for leaf nodes (degree 0).

Results

Table 1 presents the height $h(i)$ for XC3 Trees of degree $i = 0$ to 25

Derived Formula: $h(i) = \lceil i/2 \rceil$

Inspecting the data reveals a clear pattern: the height increases by 1 for every two additional degrees. Specifically: Looking at the base cases, degree 0 has a height of 0. Then, degrees 1 and 2 both share a height of 1, while degrees 3 and 4 share a height of 2. Each odd degree adds one level of height compared to the previous even degree.

$$h(i) = \left\lceil \frac{i}{2} \right\rceil$$

Explaination

The formula $h(i) = \lceil i/2 \rceil$ follows directly from the recursive structure of XC3 Trees. The height of a degree i tree is determined by its tallest subtree, which is always the last child. The last (i th) child has degree $i-2$, because by rule 3, the i th child has degree $i-2$ when $i > 2$.

This gives the recurrence relation:

$$h(i) = 1 + h(i-2), \quad \text{for } i > 2$$

With the bases cases:

$$h(0) = 0, \quad h(1) = 1, \quad h(2) = 1$$

Since each recursive call reduces i by 2, the total depth of recursion before reaching a base case is exactly $\lceil i/2 \rceil$. Therefore, the height grows by one for every two increments in degree, producing the ceiling division formula.

This means that a tree of an odd degree and the subsequent even degree share the same height (e.g., degrees 3 and 4 both have a height of 2), which is perfectly consistent with the results.

Degree i	Height $h(i)$
0	0
1	1
2	1
3	2
4	2
5	3
6	3
7	4
8	4
9	5
10	5
11	6
12	6
13	7
14	7
15	8
16	8
17	9
18	9
19	10
20	10
21	11
22	11
23	12
24	12
25	13

Table 1: Heigh of XC3 Trees for degree 0 to 25.

Conclusion

The height of a degree i XC3 Tree is $h(i) = \lceil i/2 \rceil$. While this result is straightforward, it establishes a key foundation: as the degree grows, the height grows only linearly in i. Combined with the node count pattern explored in Experiment 4, this will allow us to reason about the relationship between n (number of nodes) and height, and ultimately support an $O(\log n)$ height bound.

Experiment 4

Objective

This experiment has two goals. First, to construct XC3 Trees for degree 0 through 25, count the number of nodes in each tree,, and derive a closed formed equation $node(i)$. Secondly, to use the drived formula, together with the result from experiement 3, to construct an arguement that the height of an XC3 tree with n node is $O(\log n)$.

Experiment Setup

Using the same recusive XC3 Tree implementation from experiment 3, the number of the nodes in each tree was computed recusively as : 1 + sum of node counts of all children. Trees of degree 0 through 25 were constructed and measured.

Results

Table 2 presents $nodes(i)$ for degrees 0 through 25, alongside the corresponding Fibonacci number (Please see below)

Degree i	$nodes(i)$	**Fibonacci**
0	1	$F(2)$
1	2	$F(3)$
2	3	$F(4)$
3	5	$F(5)$
4	8	$F(6)$
5	13	$F(7)$
6	21	$F(8)$
7	34	$F(9)$
8	55	$F(10)$
9	89	$F(11)$
10	144	$F(12)$
11	233	$F(13)$
12	377	$F(14)$
13	610	$F(15)$
14	987	$F(16)$
15	1597	$F(17)$
16	2584	$F(18)$
17	4181	$F(19)$
18	6765	$F(20)$
19	10946	$F(21)$
20	17711	$F(22)$
21	28657	$F(23)$
22	46368	$F(24)$
23	75025	$F(25)$
24	121393	$F(26)$
25	196418	$F(27)$

Table 2: Node count of XC3 Trees by degree.

Pattern: Fibonacci Numbers

Examining the values in table 1 immediately reveals as a well know pattern. The sequence 1, 2, 3, 5, 8, 13, 21, 34, 55, 89 ... is exactly the Fibonacci sequence, starting from $F(2)$. The Fibonacci sequence is defined as:

$$F(1) = 1, \quad F(2) = 1, \quad F(n) = F(n-1) + F(n-2) \quad \text{for } n > 2$$

Using 1 indexed Fibonacci numbers where $F(1) = 1, F(2) = 2, F(3) = 3, F(4) = 5, \ldots$ the formula for the number of nodes in a degree i XC3 tree is:

$$nodes(i) = F(i+2)$$

where $F(k)$ denotes the k th Fibonacci number (with $F(1) = 1, F(2) = 2$).

Explaination: why does this pattern arise ?

The fibonacci pattern is a direct consequence of the recursive structure of XC3 tree. We recall that a degree i tree has children of degrees 0,0,1,2,3, $\ldots$, $(i-2)$. The total node count satisfies the recurrence:

$$nodes(i) = 1 + nodes(0) + nodes(1) + \cdots + nodes(i-2)$$

By substraction (As per suggestion from our TA)

We can also prove this by substracting the recurrence. For a tree of degree i:

$$nodes(i) = 1 + 2 \cdot nodes(0) + nodes(1) + \cdots + nodes(i-2)$$

Similarly, for a tree of degree i:

$$node(i-1) = 1 + 2 \cdot nodes(0) + nodes(1) + \cdots + nodes(i-3)$$

Subtracting the second equation from the first, we can see that most terms cancel out and result:

$$nodes(i) - nodes(i-1) = nodes(i-2)$$

Rearranging give the exact defition of the Fibonacci sequence:

$$nodes(i) = nodes(i-1) + nodes(i-2)$$

Argument : Heigh of XC3 tree is $O(\log n)$

We now combine the results of experiment 3 and 4 to argue that the height n of an XC3 tree with n nodes statisfies $h = O(\log n)$.

Step 1 — Express n in terms of degree i.
From Experiment 4, we observed that the number of nodes in a degree i tree follows the Fibonacci sequence:

$$n = nodes(i) = F(i+2)$$

Step 2 — Apply the Golden Ratio Inequality.
A well known property of Fibonacci numbers is their relationship with the Golden Ratio $\phi = (1+\sqrt{5})/2 \approx 1.618$. For any $k \geq 2$, the Fibonacci sequence satisfies the following lower bound:

$$F(k) \geq \phi^{k-2}$$

Applying this to our node count formula where $k = i + 2$:

$$n = F(i+2) \geq \phi^{(i+2)-2} = \phi^i$$

Step 3 — Solve for degree i using Logarithms.
Taking the logarithm base ϕ of both sides of the inequality $n \geq \phi^i$:

$$\log_\phi(n) \geq i$$

Since changing the base of a logarithm only involves a constant factor ($\log_\phi n = \frac{\ln n}{\ln \phi}$), where $1/\ln\phi$ is a constant, we can conclude that:

$$i = O(\log n)$$

This demonstrates that the degree i required to sustain n nodes grows logarithmically with respect to n.

Step 4 — Connect degree to height.
From Experiment 3, the height of a degree i XC3 Tree is:

$$h(i) = \left\lceil \frac{i}{2} \right\rceil$$

Substituting our bound $i = O(\log n)$ into this linear relationship:

$$h \leq \frac{i+1}{2} \implies h = O(i) = O(\log n)$$

Conclusion

By establishing the inequality $n \geq \phi^i$, we prove that the degree i grows at most as $O(\log_\phi n)$. Since the height h is a linear function of i ($h \approx i/2$), the overall height of an XC3 Tree is:

$$h(n) = O(\log n)$$

This mean XC3 Trees, despite having a more complex branching structure than binary trees, are still well balanced. Their height grow only logarithmically with the numeber of nodes. This is a desirable property shared with self balancing binary trees such as RBT, confirming that XC3 trees are an efficient tree structure.

Appendix: Code Organization

All source code for this lab is located under the `lab1/src/` directory.

- `bst.py`: Implementation of BST for experiments 1 and 2
- `rbt.py`: Implementation of RBT with completed `rotate_left(), rotate_right(), fix()`
- `experiment1.py`: Average height comparison of BST and RBT
- `experiment2.py`: Average height comparison of BST and RBT in near-sorted lists
- `experiment3.py`: Implementation of XC3-Trees with height implementation
- `experiment4.py`: Implementation of XC3-Trees with number of nodes implementation